

DriveVLN: Towards Mapless Vision-and-Language Navigation in Autonomous Driving

Dongqian Guo¹, Haoran Wei¹, Wencheng Han^{1✉},
 Runzhou Tao², Zhongying Qiu², Jianfei Yang², Jianbing Shen^{1✉}
¹SKL-IOTSC, CIS, University of Macau ²Chongqing Afari Intelligent Drive Co.,Ltd.
 {mc35099, jianbingshen}@um.edu.mo

Abstract

Autonomous driving has made substantial progress recently, achieving reliable performance in most real-world environments. However, existing algorithms still depend heavily on high-definition maps, making them ineffective in mapless scenarios such as indoor parking lots. These limitations hinder seamless point-to-point navigation and restrict the broader deployment of the autonomous driving system. To address this challenge, we propose DriveVLN, a new task that extends Vision-and-Language Navigation (VLN) to autonomous driving. DriveVLN employs visual and linguistic priors to guide vehicles toward destinations based solely on concise natural-language descriptions, without access to predefined maps or routes. Unlike conventional VLN, which relies on detailed step-wise instructions in indoor environments, DriveVLN requires models to produce navigation information based on diverse visual cues and history, including signs, landmarks, and textual indicators. We further develop a CARLA-based simulation engine comprising over 200 realistic scenes reconstructed from real road scans, enabling large-scale training and closed-loop evaluation. A baseline model is established through supervised fine-tuning on real data, followed by reinforcement learning in simulation. Comprehensive experiments show that DriveVLN effectively bridges map-based and mapless driving, providing a new foundation for unified, language-driven autonomous navigation in complex real-world environments.

1. Introduction

Autonomous driving has achieved remarkable progress in recent years [9, 11, 17, 41], enabling efficient and safe driv-

✉ Corresponding authors: Jianbing Shen, Wencheng Han. This work was supported by the Science and Technology Development Fund of Macau SAR (FDCT) under grants 0134/2025/RIA2 and 0102/2023/RIA2, and the Jiangyin Hi-tech Industrial Development Zone under the Taihu Innovation Scheme (EF2025-00003-SKL-IOTSC).

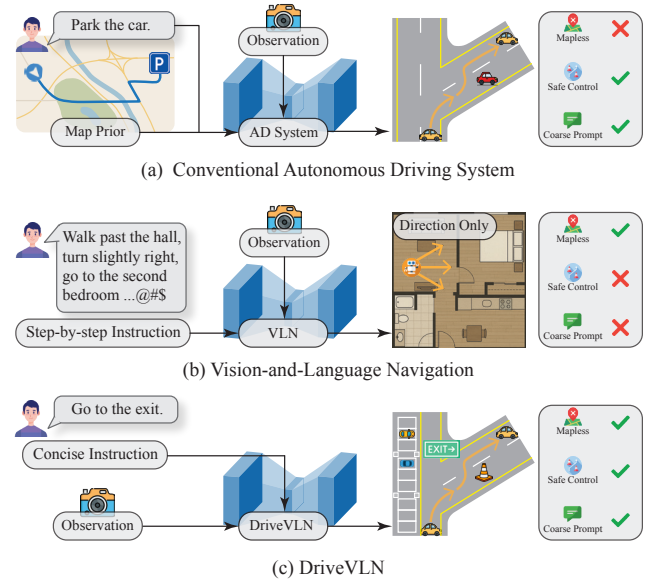


Figure 1. **Comparison of three different tasks.** (a) Most conventional autonomous driving systems rely on navigation maps as priors to realize the driver’s intent. When the destination lies in regions without reliable maps, the system often fails. (b) Vision-and-Language Navigation (VLN) was proposed to address such cases, but existing work is rooted in indoor robotics, assuming fine-grained step-by-step language supervision as priors. VLN for autonomous driving scenarios remains largely unexplored. (c) We introduce DriveVLN, a new task that achieves point-to-point navigation using only visual inputs and coarse, destination-only language prompts.

ing in most scenarios. However, there remain specific situations where general autonomous driving algorithms still fail to meet practical requirements [26, 28]. These limitations hinder the seamless point-to-point navigation expected from fully autonomous vehicles. Among these challenging situations, mapless driving scenarios represent one of the most common and critical problems yet to be solved.

As shown in Fig. 1(a), conventional autonomous driving systems heavily rely on navigation maps to plan feasible

routes and provide navigation guidance toward a destination [10, 33]. However, in mapless environments, the lack of precise geographic and route information prevents the system from determining which driving decisions should be made at each moment. Consequently, the vehicle cannot accurately infer how to move toward its goal under such uncertainty.

To address navigation in mapless settings, several algorithmic approaches have been proposed [18, 27, 30, 44], with Vision-and-Language Navigation (VLN) being one of the most representative. As illustrated in Fig. 1(b), VLN methods use natural language instructions, combined with corresponding visual observations of the environment, to infer the correct actions that lead an agent from its starting point to the target location [2, 15, 40]. While effective for indoor robotic navigation, these methods are difficult to adapt to autonomous driving tasks. A key challenge is that, in mapless driving scenarios, even humans often cannot provide detailed and precise natural language descriptions that capture the spatial relationship between the current location and the destination. In most real-world cases, drivers know where they want to go but not how to get there. This fundamental gap poses a significant obstacle to extending VLN approaches to real-world autonomous driving scenarios.

To overcome the limitations of existing methods, we propose a new task called DriveVLN, as shown in Fig. 1(c), which extends the core idea of Vision-and-Language Navigation (VLN) to autonomous driving. DriveVLN leverages both visual and linguistic priors to guide a vehicle toward its destination without relying on a pre-built map. While inspired by VLN, our task is carefully adapted to the unique requirements of real-world driving scenarios, enabling the concept of VLN to be effectively applied to mapless autonomous driving.

Specifically, DriveVLN differs from traditional VLN in three key aspects: **First**, the form of language guidance is fundamentally different. Conventional VLN tasks use detailed, step-by-step descriptions that specify key landmarks and actions along a predefined route. In contrast, DriveVLN operates without knowledge of the complete driving path. Its linguistic reference only provides a description of the destination rather than the entire route. **Second**, the target environments are distinct. In autonomous driving, especially in typical mapless settings, the environment is often rich in implicit navigational cues. These cues may appear as diverse signs or text, and experienced human drivers can interpret their spatial layout to infer the correct driving route. Accordingly, DriveVLN requires the model to develop a similar capability: understanding and leveraging these visual indicators for effective navigation. **Third**, the task execution itself is more complex. Unlike indoor navigation, DriveVLN involves actual vehicle control. It demands not only accurate route reasoning but also precise

decision-making and motion planning to ensure safe and goal-directed driving behavior.

With these design principles, DriveVLN enables autonomous vehicles to interpret destination-level natural language commands, understand their visual surroundings, and drive toward the goal without map priors. Ultimately, this framework bridges the gap between map-based and mapless driving, achieving seamless point-to-point autonomy across diverse real-world environments.

To support this new task, we developed a novel data pipeline upon the CARLA simulator [14]. The pipeline created 200 diverse driving scenarios, each constructed from real-world road scans to faithfully replicate authentic driving structures. This environment provides rich and high-fidelity data for training the DriveVLN model. In addition, we design a closed-loop benchmark for evaluation, where the vehicle is initialized at a specific position within a scene and given a textual description of the destination. The model must then safely navigate autonomously to the target location. Beyond standard autonomous-driving metrics such as L2 error and collision rate, our benchmark also incorporates VLN-specific metrics, such as arrival success rate and navigation error, to jointly assess both navigation accuracy and driving safety.

Based on this benchmark, we further introduce a baseline model for the DriveVLN task. Our DriveVLN model consists of two components: a planning module for generating trajectories and a navigation selector. Given onboard sensor inputs and without any navigation map priors, the planner enumerates a set of drivable candidate paths from the current ego state. These candidates are destination-agnostic and encode feasibility only. We then convert them into visual representations to define the decision space. The selector then consumes the language instruction, jointly reasoning over drivable-area cues and available directional indicators in the visuals. Finally selects a trajectory from this decision space.

Our contributions can be summarized as follows:

- We introduce a mapless VLN task: a new setting in which an autonomous vehicle safely reaches a destination from simple human language instructions without relying on navigation maps, filling a gap in current research on autonomous exploration under mapless conditions.
- We build a benchmark on the CARLA simulation engine, offering 200 diverse and realistic mapless roaming scenarios with an autonomous data generation pipeline. We also establish standardized metrics and evaluation protocols to provide a unified basis for training, testing, and comparing VLN methods in this setting.
- We design a new baseline model tailored to our task. The model is composed of a planning module and a navigation selector. A two-stage training strategy that combines real and simulated data to construct a well-performing base-

line model.

- We validate the baseline on our benchmark. Across different scenarios, the baseline achieves state-of-the-art performance on the constructed benchmark, demonstrating its effectiveness and feasibility for mapless autonomous parking.

2. Related Works

2.1. Language-Guided Autonomous Driving

Language-guided autonomous driving has recently emerged via open-loop evaluations and VQA-style supervision [23, 45, 46]. Wayve’s LINGO [29] series uses language to explain and train foundation driving models. DriveLM [38] introduces Graph-VQA chains that reflect reasoning from perception, prediction, and planning. NuScenes-QA [35] and LingoQA [29] supply large-scale question-answer pairs over multi-modal driving scenes. While these improve explainability and scene understanding, most are open-loop or decoupled from closed-loop control. For action-oriented, Talk2Car [13] grounds passenger commands to referred objects in nuScenes[6], and recent VLM-driving systems such as DriveVLM [43] and LMDrive [36] couple language with end-to-end policies, yet typically presume navigation map priors. Our DriveVLN pushes toward mapless, destination-level grounding with full closed-loop rollouts.

2.2. Benchmarks for Vision-and-Language Navigation

For VLN benchmarks, Room-to-Room [3] and Room-for-Room [19] evaluate instruction following via predefined step-by-step routes, using static image observations to ground the instructions. Some benchmarks are proposed targeting richer environments and decision-making recently, including CVDN [42], REVERIE [34], and SOON [48]. While these benchmarks standardize the evaluation of robotics-related tasks, they overlook the metrics commonly adopted in autonomous driving, thereby creating a substantial gap in evaluating driving-oriented VLN. The closed-loop benchmark on autonomous driving, such as Waymo Open Sim Agents Challenge [31], still cannot evaluate VLN tasks in mapless scenarios. DriveVLN closes this gap by combining navigation metrics with driving-centric evaluation to jointly measure goal-reaching accuracy and safety under destination-only language.

Some autonomous driving benchmarks are built upon specific simulators. CARLA [14] established an open, highly controllable driving simulator and Leaderboard metrics [8] for closed-loop assessment. Bench2Drive [20] proposed a closed-loop benchmark based on CARLA simulator for end-to-end autonomous driving. Additionally, data-driven simulators such as nuPlan [7], Waymax [16] and VISTA 2.0 [1] are proposed. The compositional genera-

tors like MetaDrive [24] enable large-scale training and realistic online evaluation. Our newly proposed benchmark builds on CARLA and reconstructs 200 scenes from real scans specifically for language-driven, mapless navigation and reports both driving and VLN metrics.

3. The Proposed Vision-and-Language Navigation Benchmark

We present DriveVLN, a novel benchmark built within the CARLA simulator for vision-language navigation (VLN) in autonomous driving under sparse prior information and mapless conditions. DriveVLN benchmark formalizes the VLN task in autonomous driving and provides a dataset that comprises 200 high-fidelity scenes. These digital-twin scenes are aligned well with the real world, primarily targeting environments where navigation maps are often unavailable, such as indoor parking lots. The dataset also with complete routes from a random starting point to the destination and textual task descriptions. To construct the dataset, we develop a CARLA-based data pipeline with strong flexibility, extensibility, and reproducibility, which enables large-scale data collection and simulation. We further define a suite of evaluation metrics to enable fair and consistent comparison across diverse baseline models.

3.1. Task Definition

In general, driving with natural language navigation requires the model to output an executable trajectory w_t towards the correct direction at the current time step t based on a given text instruction T and the input RGB image I . We formalize the proposed task as an option-based Partially Observable Markov Decision Process (POMDP) [21, 22, 39].

We first define the state space $\mathcal{S} = \{s_0, s_1, \dots, s_t\}$ of the ego vehicle. At each time t , the posture x_t and the driving command c_t of the ego vehicle are obtained by sensors. The success of the VLN task directly depends on whether the destination has been reached. Therefore, we introduce $g_t \in \{0, 1\}$ to the state space as an arrival flag, where $g_t = 1$ indicates arrival and $g_t = 0$ otherwise. This flag serves as the termination condition for the agent. Consequently, the ego state at time step t is defined as $s_t = (x_t, c_t, g_t)$. The observation o_t is generated from the rendered image $I_t \sim \mathcal{O}(\cdot | s_t)$, where $\mathcal{O}$ denotes the simulator’s rendering process. A sequence of such observations o_t constitutes the observation space $\mathcal{Z}$. At each decision step, the VLN agent conditions its decisions on the history $h_t = (T, o_{0:t}, a_{0:t-1})$. A planner generates K candidate trajectories based on the current observation as the action space:

$$\mathcal{A}_t = \{\omega_t^k\}_{k=1}^K \sim \mathcal{P}(o_t)$$

where $\mathcal{P}$ denotes the planner and each candidate trajectory

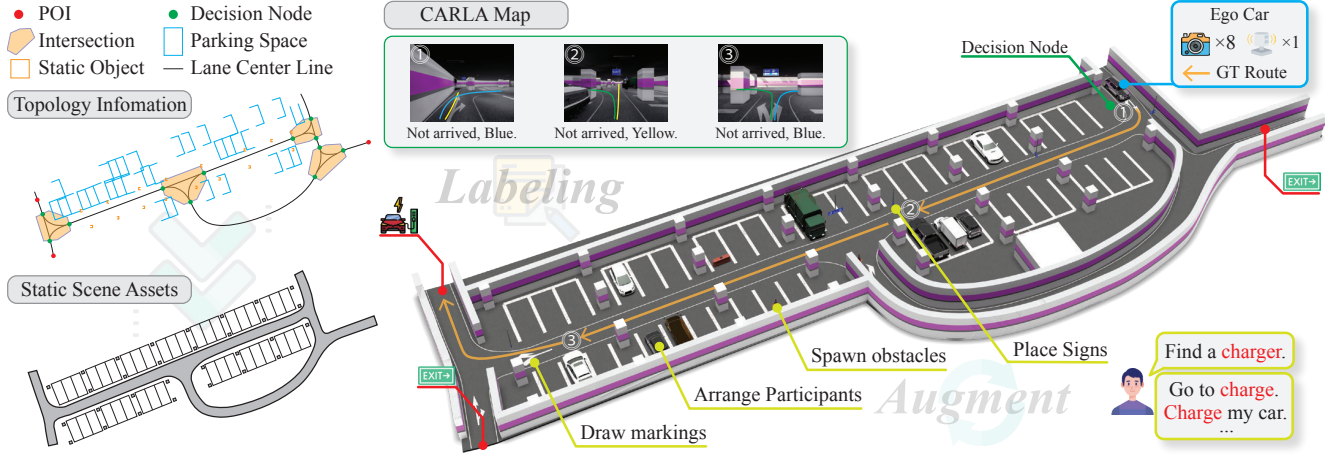


Figure 2. **Overview of the Data Generation Pipeline.** (a) We first generate static scene assets from the collected topology information. Then, repair and rearrange the coarse-grained detected elements. (b) We perform text and scene augmentation. Conditioned on user-intent keywords, we synthesize diverse instruction paraphrases. In the environment, we apply rule-based stochastic placement of traffic participants and obstacles and procedurally render lane markings and traffic signs. (c) Finally, we automatically record and annotate data along predefined routes in the CARLA simulator.

ω_t^k consists of L waypoints. The selector chooses one trajectory using policy $\pi_\theta(\omega \mid h_t, \mathcal{A}_t)$ and the simulator κ executes it for L steps to yield controls $a_{t:t+L-1}$. When $\mathcal{A}_t$ degenerates into a single externally supplied fine-grained route with deterministic κ , the formulation collapses to conventional VLN execution and evaluation.

3.2. Data Generation Pipeline

We propose a Topo2Sim data generation pipeline for autonomous driving built on CARLA simulator. The pipeline consists of three stages. First, the pipeline generates the scene assets from topology information collected by vehicles, then augments the textual instructions and scene elements. Finally, we conduct data collection with automatic labeling in the simulator.

3.2.1. Generate Scene Assets

Scene topology information originates from drive data collected in real environments. During preprocessing, upstream models detect road centerlines, the geometries and connectivity patterns of junctions, parking slots, and static obstacles such as pillars and walls, and they log a series of points of interest (POIs). These POIs are typical user destinations such as exits and EV chargers. All topological entities are represented in geodetic coordinates and stored in a structured file. The asset generation process can be divided into three stages.

Coordinate Normalization. All topology coordinates are provided as raw WGS84 geodetic coordinates. We first convert them to metric units, then apply a unified transform to the simulator’s coordinate frame. Finally, we remove salient outliers and re-sample each edge at uniform spacing, enabling robust downstream post-processing.

Lane Boundary Generation. For each lane centerline C_e , we first parameterize it by arc length s .

$$C_e : s \mapsto (x(s), y(s))$$

Using the Frenet frame, we define at position s the unit normal vector $\mathbf{N}(s)$ pointing laterally from the centerline. We then compute the left and right road boundaries as

$$B^\pm(s) = C_e(s) \pm \left(\frac{1}{2}w(s)\right)\mathbf{N}(s)$$

where $B^\pm(s)$ represents the positions of each side lane boundary at arc s , and $w(s)$ denotes the lane width.

Junction continuity and connectivity maintenance. We detect shared endpoints across polylines and enforce consistent parameterization and boundary alignment at those shared points. Then, we populate directed predecessors and successors based on endpoint matching and local tangent compatibility. This step turns the lanes into a directed multigraph suitable for routing in the simulator.

3.2.2. Autonomous Augmentation and Data Collection

Our augmentation strategy has two components: text augmentation and scene augmentation. For each POI, we leverage GPT-4 to derive keyword-conditioned paraphrases, yielding semantically equivalent but lexically diverse navigation instructions for training and thereby improving the model’s robustness. For scene augmentation, we exploit the simulator’s flexibility. Through rule-based stochastic procedures, we populate environments with parked vehicles, other traffic participants, and elements such as lane markings, obstacles, and traffic signs. This enables multiple realizations of a scene to be generated from the same topology, increasing data diversity during collection.

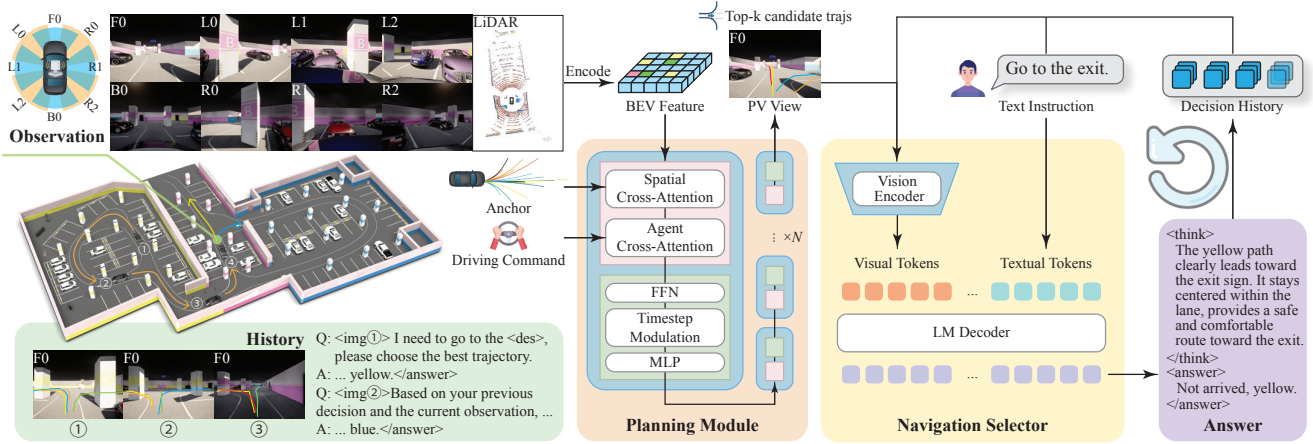


Figure 3. **Overview of the DriveVLN Model Framework.** Our framework has two components. The **Planning Module** encodes on-board sensor data and generates feasible driving trajectories based on trajectory anchors. The **Navigation Selector** is built upon a VLM architecture, which is responsible for selecting a reasonable path among multiple feasible candidates that leads toward the destination.

Under our task definition, the VLN decomposes into two sub-tasks. The first one is deciding whether the ego vehicle has reached the destination, which is the termination criterion. The second one is issuing navigation signals that steer the vehicle toward the goal. On straight road segments, the primary objective is arrival checking. When approaching an intersection, the policy must select the correct branch conditioned on the language instruction and the visual observation. Accordingly, we fuse lane centerlines with intersection geometry to construct a road-topology decision graph $\mathcal{G} = (v, e)$. We define a decision node as the ego pose immediately before a junction and oriented toward it. Each decision node connects to several candidate branches, each leading to the next decision node. These branches are instantiated by lane-centerline segments that compose into polylines and serve as graph edges. Edge lengths are metrically consistent with the physical road network, yielding a faithful reconstruction of the real scene.

During data collection, we align our logging protocol with NAVSIM[12]. In CARLA, we mount eight RGB cameras and one LiDAR on a data-collection vehicle. We randomly select a decision node as the start and a POI as the destination, and compute the shortest path on $\mathcal{G}$ using physical road length as the edge weight, yielding the optimal route. Beginning at the start node, we record at 2 Hz along straight road segments. At each decision node, we additionally capture a keyframe to mark the junction, projecting all feasible paths into the front camera to supervise VLN training. We also log the ego vehicle’s control commands and executed trajectory from the simulator’s motion planner, enabling training of the planning module.

4. DriveVLN Model

4.1. Model Framework

Our DriveVLN model comprises a planning module and a navigation selector. The planner proposes a set of traversable but destination-agnostic candidate trajectories based on the given current multi-sensor observations. The selector handles decision-making and chooses a comfortable and safe trajectory. Conditioned on visual context, candidate trajectories, and language instruction, it chooses the option that best advances toward the destination.

Planning Module. The planning module is a real-time trajectory generator which is instantiated using Diffusion-Drive [25] as a baseline. Given egocentric RGB and LiDAR data as conditions, use the anchored Gaussian distribution as prior to sample some noisy trajectories. Then the planning module performs a denoise process. We select the *top-k* outputs as feasible driving trajectories under the current state, defining the selector’s action space $\mathcal{A}_t$. To be consistent with NAVSIM, each output trajectory w_t has eight waypoints over a four-second horizon.

Navigation Selector. As illustrated in Fig. 3, we use an instruction-following VLM fine-tuned from Qwen2.5-VL-3B [4] as the navigation selector. We first rasterize the candidate trajectories onto the forward camera image with distinct colors. We then feed the resulting composite together with the language instruction into the model. The selector first determines whether the destination has been reached. If not, it selects a candidate path by answering the color corresponding to that path as the navigation action. The corresponding trajectory is then passed to the simulator, which computes a series of metrics. To preserve temporal context, the selector maintains a rolling history of visual input and past decisions by QA turns, enabling decisions that are consistent with the route taken so far.

4.2. Training Strategy

Trajectory Foundry. First, we finetune a well-performed planner module. We sample random routes from augmented scenes and use the simulator to generate ground truth trajectories. The start and end points are randomly set so that the trajectories are without destination-related priors. Then, we obtain the anchors by clustering all recorded trajectories. The planner learns to propose drivable yet destination-agnostic trajectories.

Second, we perform instruction tuning on the selector using real-world collected discrete single-frame data. The POI description is augmented and concatenated with a pre-defined prompt template, while the model output is constrained to the fixed format:“(arrived or not);(color.i)...”

Through supervised training and instruction tuning, the planner acquires basic trajectory-generation ability, while the selector learns the QA behavior and output format. Notably, the selector may return multiple candidate paths at this stage. Under limited priors, routes to a destination can be non-unique. Next, we introduce **Policy Tempering** to further disambiguate and reduce such multiplicities.

Policy Tempering. During this stage, we focus training on optimizing the navigation selector. We conduct reinforcement learning so that it learns to exploit its own decision history to compensate for missing priors at the current timestep. For example, when the agent arrives at an intersection with no navigation cue, the policy is encouraged to rely on accumulated context and experience to choose the correct branch.

We randomly initialize a start node s and set the goal g to a sampled POI on $\mathcal{G}$. During navigation, the selector emits M discrete decisions, producing the visited decision-node sequence $\hat{\mathbf{v}}_{0:M} = (\hat{v}_0 = s, \hat{v}_1, \dots, \hat{v}_M)$. The edge weight between two decision nodes is given by the physical length of the corresponding lane-centerline segments from the scene topology information. Let the optimal route be $\mathbf{v}_{0:K} = (v_0 = s, v_1, \dots, v_K = g)$, with total geometric length $L = \sum w(e)$. At decision time t , we denote the selector’s chosen trajectory by $\hat{\tau}_t = \{\hat{\mathbf{p}}_{t,i}\}_{i=1}^L$, and the ground-truth trajectory by $\tau_{gt} = \{\mathbf{p}_{t,i}\}_{i=1}^L$. Both segments are arc-length resampled to share the same L waypoints. We use a binary indicator $\text{Col}_t \in \{0, 1\}$ to record whether any collision occurs after executing the navigation choice at time t . Naturally, the reward $\mathcal{R}$ comprises two terms.

The first term computes a local reward that accounts for trajectory comfort and safety. Comfort measures the alignment between the selected and the ground truth trajectory. We take the mean distance between the L waypoints and normalize it to $[0, 1]$:

$$S_t^{\text{match}} = \exp\left(-\frac{1}{L} \sum_{i=1}^L \min\|\hat{\mathbf{p}}_{t,i} - \mathbf{p}_{t,i}\|_2\right) \quad (1)$$

Safety is implemented as a gate $S_t^{\text{saf}} = 1 - \text{Col}_t$, so any collision at step t zeroes the score:

$$r_t^{\text{local}} = S_t^{\text{saf}} \cdot S_t^{\text{match}} \quad (2)$$

The per-step local reward and its episode-level aggregation over M selector decisions are

$$\mathcal{R}_{\text{local}} = \frac{1}{M} \sum_{t=1}^M r_t^{\text{local}} \quad (3)$$

The second term is a global reward that is computed only after M rounds of interaction. It measures how consistent the whole route induced by the selector from the start node is with the shortest route. First, we compare the chosen route with the optimal route on equal-length prefixes to count whether the same branch is taken at each time step:

$$S_{\text{prefix}} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}[\hat{v}_t = v_t], T = \min(M, K) \quad (4)$$

Next, we compute the edge-overlap ratio between the chosen route and the shortest route. Let $\hat{\varepsilon} = \{(v_{t-1}, v_t)\}_{t=1}^M$ be the set of edges traversed by the model, and let ε be the edge set of the prefix of the optimal path.

$$S_{\text{overlap}} = \frac{|\hat{\varepsilon} \cap \varepsilon|}{\max(1, |\varepsilon|)} \quad (5)$$

The global score is finally defined as

$$\mathcal{R}_{\text{global}} = \alpha_1 S_{\text{prefix}} + \alpha_2 S_{\text{overlap}} + \alpha_3 \mathbf{1}[\text{arrived}] \quad (6)$$

The learning objective is to maximize the discounted return:

$$J(\theta) = \mathbb{E}\left[\sum_{t=0}^t \gamma^t \mathcal{R}_t\right] \quad (7)$$

$$\mathcal{R}_t = \lambda \mathcal{R}_{\text{local}} + (1 - \lambda) \mathcal{R}_{\text{global}} \quad (8)$$

where $\gamma \in (0, 1]$ indicates the discount factor, λ used to balance the rewards, and $\mathbf{1}[\cdot]$ provides a reward to the model when it successfully arrives at the destination.

At step t , the planner proposes $\mathcal{A}_t$. The selector, which is our policy, chooses an option $w_t \sim \pi_\theta(\cdot | h_t, \mathcal{A}_t)$. Let $\bar{r}_t = \sum_{j=0}^{L-1} \gamma^j r_{t+j}$ and h_{t+L} be the reward and history after executing the choice. We add a term to Equation (11) that represents an KL prior to an SFT policy π_0 , then the episodic objective becomes:

$$\begin{aligned} \max_{\theta} J(\theta) = & \mathbb{E}\left[\sum_{t=0}^T \gamma^t \mathcal{R}_t\right] \\ & - \beta \mathbb{E}\left[\text{KL}(\pi_\theta(\cdot | h_t, \mathcal{A}_t) \parallel \pi_0(\cdot | h_t, \mathcal{A}_t))\right]. \end{aligned} \quad (9)$$

Then we apply the policy gradient theorem, and the gradient depends on the selector only:

$$\nabla_{\theta} J = \mathbb{E} \left[\sum_{t=0}^t \nabla_{\theta} \log \pi_{\theta}(\omega_t | h_t, \mathcal{A}_t) A_t \right] \quad (10)$$

Using the old policy $\pi_{\theta}^{\text{old}}$, we sample N rollouts, each consisting of M interaction rounds, and calculate the reward $\{R_n\}_{n=1}^N$. The advantage term A_t can be estimated using any effective method from reinforcement learning. In our implementation, we follow best practice and adopt Group Relative Policy Optimization (GRPO) [37] as the advantage estimator:

$$\hat{A}_n = \frac{R_n - \frac{1}{N} \sum_j R_j}{\sqrt{\frac{1}{N} \sum_j (R_j - \bar{R})^2 + \beta}}, \bar{R} = \frac{1}{N} \sum_j R_j \quad (11)$$

Finally, we compute the GRPO loss $\mathcal{L}_{\text{GRPO}}$ and update the policy parameters θ .

5. Experiment

5.1. Evaluation Metrics

We couple the reward design used during Policy Tempering with an evaluation suite that draws from classical VLN (navigation success and error rates) and driving-specific metrics (collision rate and L2 error).

Following the closed-loop evaluation protocol of the CARLA Leaderboard [8], the primary metric is the Driving Score D , defined as route completion multiplied by an infraction penalty:

$$D = RC \cdot IP \quad (12)$$

IP is derived from our local reward. Over the full path obtained after M interaction rounds, we count the total number of collision events N_{col} and map it into $(0, 1]$ via an exponential normalization:

$$P_{\text{col}} = \exp(-N_{\text{col}}) \quad (13)$$

For each time step, we then define a deviation threshold:

$$\Delta_t = \max(0, 1 - S_t^{\text{match}} - \beta)$$

β is the tolerance we set to 0.2, and we calculate the deviation penalty:

$$P_{\text{dev}} = \exp\left(-\frac{1}{M} \sum_t \Delta_t\right) \quad (14)$$

Finally, we get the Infraction Penalty:

$$IP = P_{\text{col}} \cdot P_{\text{dev}} \quad (15)$$

RC is derived from our global reward. We first define a navigation accuracy:

$$Q = \omega S_{\text{prefix}} + (1 - \omega) S_{\text{overlap}}, \omega \in [0, 1]$$

Then the $SR = \mathbf{1}[\text{arrived}]$, $NE = 1 - Q$, and calculate the route completion:

$$RC = SR + (1 - SR) \cdot (1 - NE) \quad (16)$$

5.2. Implementation Details

We conduct experiments on eight NVIDIA A800 GPUs. During the Trajectory Foundry phase, we implemented the planner using the official code of DiffusionDrive [25], with both training and inference following its original settings. All training data are collected from the DriveVLN benchmark, based on the CARLA simulator. The dataset contains a total of 352,230 frames and is trained for 50 epochs.

While performing LoRA SFT on the selector, we use LlamaFactory [47], with the learning rate set to 1×10^{-4} , the LoRA rank set to 64, and the batch size per GPU set to 4. We trained using a total of 40,780 frames of images, with each image having a resolution of 800×600 . The total training spanned 5 epochs.

In the Policy Tempering phase, we set $M = 8, N = 10$, which means we rollout eight times per round and each rollout has ten interactions. The weight of the KL loss was set to 0.001, the learning rate was set to 1×10^3 with a cosine annealing strategy, and training was also conducted for 5 epochs.

5.3. Experimental Result Analysis

We evaluate different baseline models as the navigation selector on the DriveVLN benchmark. The baseline models include state-of-the-art proprietary multimodal VLMs such as Seed-1.6 [5] and GPT-5 [32], as well as a widely used open-source VLM model, Qwen2.5-VL-72B [4]. We randomly sample 20 preset routes across the augmented scenes. At test time, the planning module proposes feasible trajectories. We take the top-4 trajectories by confidence, project them to the perspective view, and pass them to the navigation selector. Following Sec. 5.1, we compute the L2 error and collision rate for each frame to assess the safety and comfort. At each decision node, we record the navigation choice to derive the success rate and navigation error for VLN competence. These four metrics are aggregated into the Driving Score, which ranges from 0 to 1. As shown in Tab. 1, our DriveVLN achieves the best overall performance with a Driving Score of 0.67, followed by Seed-1.6 [5] at 0.60. Both maintain a low L2 error and collision rate, indicating safe and comfortable choices. Nonetheless, due to task difficulty, all methods achieve arrival rates below 0.5, suggesting substantial headroom for improvement on the DriveVLN task.

Table 1. **Quantitative Evaluation.** Comparison with different models on the DriveVLN benchmark.

Models	DS $\uparrow$	SR $\uparrow$	NE $\downarrow$	L2(m) $\downarrow$				Col. Rate(%) $\downarrow$			
				1s	2s	3s	Avg.	1s	2s	3s	Avg.
Seed-1.6 [5]	0.60	0.38	0.56	0.121	0.249	0.391	0.316	0.084	0.090	0.093	0.101
GPT-5 [32]	0.48	0.23	0.56	0.448	1.190	2.132	1.550	0.095	0.115	0.140	0.170
Qwen2.5-VL-72B [4]	0.45	0.21	0.59	0.505	1.396	2.537	1.868	0.108	0.143	0.191	0.240
DriveVLN(Ours)	0.67	0.44	0.49	0.136	0.223	0.344	0.283	0.088	0.090	0.094	0.096

Table 2. **Evaluation on Real World Data.** We evaluate VLN performance of different models on real-world data by selecting the three most frequent destinations. *arr.* reports arrival-detection accuracy, while *navi.* reports trajectory-selection accuracy.

Models	Parking Slot		EV Charger		Exit&Entrance		Overall	
	<i>arr.</i>	<i>nav.</i>	<i>arr.</i>	<i>nav.</i>	<i>arr.</i>	<i>nav.</i>	<i>arr.</i>	<i>nav.</i>
Seed-1.6 [5]	59.42	48.66	52.83	52.83	50.92	50.51	56.15	49.68
GPT-5 [32]	81.31	56.28	69.27	57.14	87.83	63.19	81.89	58.43
Qwen2.5-VL-72B [4]	1.34	50.41	32.08	52.56	87.83	44.79	30.51	48.98
DriveVLN (Ours)	98.61	68.54	58.76	58.49	89.47	56.24	91.40	63.75

We further assess several baselines on real-world production data in a single-turn question and answer setting. We manually annotate the drivable paths, prompt each model toward three distinct destinations, including the parking slot, the EV charger, and the exit&entrance. We report accuracy for two fundamental capabilities on discrete frames: arrival detection and navigation path selection. As shown in Tab. 2, our model attains state-of-the-art results on nearly all items. Qwen2.5-VL-72B [4] underperforms on detecting arrival at the parking slots, which only has 1.34% accuracy, whereas GPT-5 demonstrates strong generalization, outperforming other baseline models under the long-tail distribution characteristic of parking lots.

5.4. Ablation Studies

We ablate the two-stage training strategy and the choice of planner module. The results are summarized in Tab. 3. With only supervised fine-tuning in the Trajectory Foundry stage, the model makes reasonable single-frame navigation decisions. However, its navigation performance in mapless settings degrades because it under-utilizes historical context and lacks robust environmental priors. Under limited prior cues, especially when visual guidance is missing, the driving score is 0.49, and the success rate is 0.21. After reinforcement learning in the Policy Tempering stage, the DriveVLN module acquires stronger global route awareness and exhibits more “intuitive” judgments at inference. Using the local reward, it prefers safer and more comfortable trajectories and filters out the clearly erroneous candidates. When expert trajectories are used as planner outputs, collisions and L2 error are zero by construction and are therefore omitted from Tab. 3. Compared to DiffusionDrive outputs, expert trajectories are more regular in PV view and less noisy and closer to the SFT training distribution. Therefore, this setting yields a slight performance gain. Overall, regardless of the planner module, Policy Tempering substantially improves DriveVLN performance.

Table 3. **Ablation Study.** We present ablation results across training stages and planning module choices. **DD** indicates the DiffusionDrive model.

Stage	Planner	DS $\uparrow$	SR $\uparrow$	NE $\downarrow$	L2 $\downarrow$	Col $\downarrow$
SFT	DD	0.49	0.21	0.53	0.457	0.107
SFT+RL	DD	0.67	0.44	0.49	0.283	0.096
SFT	Expert	-	0.23	0.48	-	-
SFT+RL	Expert	-	0.51	0.42	-	-

6. Conclusion

In this work, we introduced DriveVLN, a new vision-and-language navigation task designed for autonomous driving in mapless environments. By extending the principles of VLN beyond indoor robotics, DriveVLN enables vehicles to interpret natural-language destination descriptions and navigate safely without relying on high-definition maps. We developed a large-scale CARLA-based benchmark containing 200 realistic driving scenes, along with a closed-loop benchmark that jointly evaluates driving safety and navigation accuracy. To establish a starting point for future research, we proposed a baseline model that integrates supervised fine-tuning on real driving data with reinforcement learning in simulation, demonstrating the feasibility of language-guided driving control. Experimental results show that DriveVLN effectively bridges map-based and mapless driving, supporting seamless point-to-point autonomy. In the future, we plan to expand DriveVLN toward larger and more diverse real-world datasets, explore large-scale language models for more robust reasoning and generalization. We believe that DriveVLN lays an essential foundation for next-generation autonomous driving systems that combine perception, language understanding, and decision-making in a unified framework.

References

- [1] Alexander Amini, Tsun-Hsuan Wang, Igor Gilitschenski, Wilko Schwarting, Zhijian Liu, Song Han, Sertac Karaman,

- and Daniela Rus. Vista 2.0: An open, data-driven simulator for multimodal sensing and policy learning for autonomous vehicles. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 2419–2426. IEEE, 2022. 3
- [2] Dong An, Yuankai Qi, Yangguang Li, Yan Huang, Liang Wang, Tieniu Tan, and Jing Shao. Bevbort: Multimodal map pre-training for language-guided navigation. *arXiv preprint arXiv:2212.04385*, 2022. 2
- [3] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton Van Den Hengel. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3674–3683, 2018. 3
- [4] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 5, 7, 8
- [5] ByteDance Seed Team. Introduction to techniques used in seed1.6. <https://seed.bytedance.com/>, 2025. Includes Seed1.6-Thinking variant overview. 7, 8
- [6] Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020. 3
- [7] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021. 3
- [8] CARLA Team and Embodied AI Foundation. Carla autonomous driving leaderboard, 2025. Version 2.1; accessed Oct. 16, 2025. 3, 7
- [9] Ming-Fang Chang, John Lambert, Patsorn Sangkloy, Jagjeet Singh, Slawomir Bak, Andrew Hartnett, De Wang, Peter Carr, Simon Lucey, Deva Ramanan, et al. Argoverse: 3d tracking and forecasting with rich maps. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8748–8757, 2019. 1
- [10] Howard Chen, Alane Suhr, Dipendra Misra, Noah Snaveley, and Yoav Artzi. Touchdown: Natural language navigation and spatial reasoning in visual street environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12538–12547, 2019. 2
- [11] Shaoyu Chen, Bo Jiang, Hao Gao, Bencheng Liao, Qing Xu, Qian Zhang, Chang Huang, Wenyu Liu, and Xinggang Wang. Vadv2: End-to-end vectorized autonomous driving via probabilistic planning. *arXiv preprint arXiv:2402.13243*, 2024. 1
- [12] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, et al. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. *Advances in Neural Information Processing Systems*, 37:28706–28719, 2024. 5
- [13] Thierry Deruyttere, Simon Vandenhende, Dusan Grujicic, Luc Van Gool, and Marie-Francine Moens. Talk2car: Taking control of your self-driving car. *arXiv preprint arXiv:1909.10838*, 2019. 3
- [14] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. Carla: An open urban driving simulator. In *Conference on robot learning*, pages 1–16. PMLR, 2017. 2, 3
- [15] Jing Gu, Eliana Stefani, Qi Wu, Jesse Thomason, and Xin Eric Wang. Vision-and-language navigation: A survey of tasks, methods, and future directions. *arXiv preprint arXiv:2203.12667*, 2022. 2
- [16] Cole Gulino, Justin Fu, Wenjie Luo, George Tucker, Eli Bronstein, Yiren Lu, Jean Harb, Xinlei Pan, Yan Wang, Xiangyu Chen, et al. Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research. *Advances in Neural Information Processing Systems*, 36:7730–7742, 2023. 3
- [17] Wencheng Han, Dongqian Guo, Cheng-Zhong Xu, and Jianbing Shen. Dme-driver: Integrating human decision logic and 3d scene perception in autonomous driving. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 3347–3355, 2025. 1
- [18] Karl Moritz Hermann, Mateusz Malinowski, Piotr Mirowski, Andras Banki-Horvath, Keith Anderson, and Raia Hadsell. Learning to follow directions in street view. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 11773–11781, 2020. 2
- [19] Vihan Jain, Gabriel Magalhaes, Alexander Ku, Ashish Vaswani, Eugene Ie, and Jason Baldridge. Stay on the path: Instruction fidelity in vision-and-language navigation. *arXiv preprint arXiv:1905.12255*, 2019. 3
- [20] Xiaosong Jia, Zhenjie Yang, Qifeng Li, Zhiyuan Zhang, and Junchi Yan. Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. In *NeurIPS 2024 Datasets and Benchmarks Track*, 2024. 3
- [21] Leslie Pack Kaelbling. Learning policies for partially observable environments: Scaling up. In *Machine Learning Proceedings 1995: Proceedings of the Twelfth International Conference on Machine Learning, Tahoe City, California, July 9-12 1995*, page 362. Morgan Kaufmann, 2014. 3
- [22] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998. 3
- [23] Jinkyu Kim, Anna Rohrbach, Trevor Darrell, John Canny, and Zeynep Akata. Textual explanations for self-driving vehicles. In *Proceedings of the European conference on computer vision (ECCV)*, pages 563–578, 2018. 3
- [24] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. 3
- [25] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, et al. Diffusiondrive: Truncated diffusion model

- for end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12037–12047, 2025. 5, 7
- [26] Henry X Liu and Shuo Feng. Curse of rarity for autonomous vehicles. *nature communications*, 15(1):4808, 2024. 1
- [27] Xinhao Liu, Jintong Li, Yicheng Jiang, Niranjana Sujay, Zhicheng Yang, Juexiao Zhang, John Abanes, Jing Zhang, and Chen Feng. Citywalker: Learning embodied urban navigation from web-scale videos. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 6875–6885, 2025. 2
- [28] Dongyuan Lu, Haoyang Du, Zhengfei Wu, and Shuo Yang. Risk assessment in autonomous driving: a comprehensive survey of risk sources, methodologies, and system architectures. *Autonomous Intelligent Systems*, 5(1):24, 2025. 1
- [29] Ana-Maria Marcu, Long Chen, Jan Hünemann, Alice Karnsund, Benoit Hanotte, Prajwal Chidananda, Saurabh Nair, Vijay Badrinarayanan, Alex Kendall, Jamie Shotton, et al. Lingoqa: Visual question answering for autonomous driving. In *European Conference on Computer Vision*, pages 252–269. Springer, 2024. 3
- [30] Piotr Mirowski, Matt Grimes, Mateusz Malinowski, Karl Moritz Hermann, Keith Anderson, Denis Teplyashin, Karen Simonyan, Andrew Zisserman, Raia Hadsell, et al. Learning to navigate in cities without a map. *Advances in neural information processing systems*, 31, 2018. 2
- [31] Nico Montali, John Lambert, Paul Mougin, Alex Kuefler, Nicholas Rhinehart, Michelle Li, Cole Gulino, Tristan Emrich, Zoey Yang, Shimon Whiteson, et al. The waymo open sim agents challenge. *Advances in Neural Information Processing Systems*, 36:59151–59171, 2023. 3
- [32] OpenAI. Introducing gpt-5. <https://openai.com/index/introducing-gpt-5/>, 2025. Product announcement and overview. 7, 8
- [33] Teddy Ort, Liam Paull, and Daniela Rus. Autonomous vehicle navigation in rural environments without detailed prior maps. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 2040–2047. IEEE, 2018. 2
- [34] Yuankai Qi, Qi Wu, Peter Anderson, Xin Wang, William Yang Wang, Chunhua Shen, and Anton van den Hengel. Reverie: Remote embodied visual referring expression in real indoor environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9982–9991, 2020. 3
- [35] Tianwen Qian, Jingjing Chen, Linhai Zhuo, Yang Jiao, and Yu-Gang Jiang. Nuscenes-qa: A multi-modal visual question answering benchmark for autonomous driving scenario. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4542–4550, 2024. 3
- [36] Hao Shao, Yuxuan Hu, Letian Wang, Guanglu Song, Steven L Waslander, Yu Liu, and Hongsheng Li. Lmdrive: Closed-loop end-to-end driving with large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15120–15130, 2024. 3
- [37] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 7
- [38] Chonghao Sima, Katrin Renz, Kashyap Chitta, Li Chen, Hanxue Zhang, Chengen Xie, Jens Beilwenger, Ping Luo, Andreas Geiger, and Hongyang Li. Drivelm: Driving with graph visual question answering. In *European conference on computer vision*, pages 256–274. Springer, 2024. 3
- [39] Richard D Smallwood and Edward J Sondik. The optimal control of partially observable markov processes over a finite horizon. *Operations research*, 21(5):1071–1088, 1973. 3
- [40] Xinshuai Song, Weixing Chen, Yang Liu, Weikai Chen, Guanbin Li, and Liang Lin. Towards long-horizon vision-language navigation: Platform, benchmark and method. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12078–12088, 2025. 2
- [41] Wenchao Sun, Xuewu Lin, Yining Shi, Chuang Zhang, Hao-ran Wu, and Sifa Zheng. Sparsedrive: End-to-end autonomous driving via sparse scene representation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8795–8801. IEEE, 2025. 1
- [42] Jesse Thomason, Michael Murray, Maya Cakmak, and Luke Zettlemoyer. Vision-and-dialog navigation. In *Conference on Robot Learning*, pages 394–406. PMLR, 2020. 3
- [43] Xiaoyu Tian, Junru Gu, Bailin Li, Yicheng Liu, Yang Wang, Zhiyong Zhao, Kun Zhan, Peng Jia, Xianpeng Lang, and Hang Zhao. Drivelm: The convergence of autonomous driving and large vision-language models. *arXiv preprint arXiv:2402.12289*, 2024. 3
- [44] Arun Balajee Vasudevan, Dengxin Dai, and Luc Van Gool. Talk2nav: Long-range vision-and-language navigation in cities. *CoRR*, 2019. 2
- [45] Shihao Wang, Zhiding Yu, Xiaohui Jiang, Shiyi Lan, Min Shi, Nadine Chang, Jan Kautz, Ying Li, and Jose M Alvarez. Omnidrive: A holistic vision-language dataset for autonomous driving with counterfactual reasoning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 22442–22452, 2025. 3
- [46] Zhiyuan Zhang, Xiaofan Li, Zhihao Xu, Wenjie Peng, Zijian Zhou, Miaoqing Shi, and Shuangping Huang. Mpdribe: Improving spatial understanding with marker-based prompt learning for autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12089–12099, 2025. 3
- [47] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyuan Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. 7
- [48] Fengda Zhu, Xiwen Liang, Yi Zhu, Qizhi Yu, Xiaojun Chang, and Xiaodan Liang. Soon: Scenario oriented object navigation with graph-based exploration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12689–12699, 2021. 3